

MVLU PRACTICAL NO:5

AIM:Key Exchange using Diffie-Hellman: Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.

```
# =====
# DIFFIE-HELLMAN KEY EXCHANGE
# STEP-BY-STEP COMMAND LINE PROGRAM
# =====

print("=" * 60)
print("    DIFFIE-HELLMAN KEY EXCHANGE")
print("=" * 60)

# STEP 1: Public Parameters
print("\nSTEP 1: PUBLIC PARAMETERS")

p = int(input("Enter prime number (p): "))
g = int(input("Enter generator (g): "))

print("\np =", p)
print("g =", g)

# STEP 2: Private Keys
print("\n\nSTEP 2: PRIVATE KEYS")

sanjana_private = int(input("Enter Sanjana's private key: "))
priya_private = int(input("Enter Priya's private key: "))

print("\nSanjana's Private Key =", sanjana_private)
print("Priya's Private Key  =", priya_private)

# STEP 3: Sanjana Public Key
print("\n\nSTEP 3: SANJANA GENERATES PUBLIC KEY")

sanjana_public = pow(g, sanjana_private, p)

print("\nFormula:")
print("Sanjana Public Key = g^a mod p")

print("\nCalculation:")
print("Sanjana Public Key =", g, "^", sanjana_private, "mod", p)

print("\nSanjana Public Key =", sanjana_public)

# STEP 4: Priya Public Key
print("\n\nSTEP 4: PRIYA GENERATES PUBLIC KEY")

priya_public = pow(g, priya_private, p)

print("\nFormula:")
print("Priya Public Key = g^b mod p")

print("\nCalculation:")
print("Priya Public Key =", g, "^", priya_private, "mod", p)

print("\nPriya Public Key =", priya_public)

# STEP 5: Public Key Exchange
print("\n\nSTEP 5: PUBLIC KEY EXCHANGE")

print("\nSanjana -> Priya :", sanjana_public)
print("Priya -> Sanjana :", priya_public)

print("\nPrivate keys are NOT exchanged.")
```

```

# STEP 6: Sanjana Shared Secret
print("\n\nSTEP 6: SANJANA CALCULATES SHARED SECRET")

sanjana_shared = pow(priya_public, sanjana_private, p)

print("\nSanjana Shared Secret =", priya_public, "^", sanjana_private, "mod", p)

print("\nSanjana Shared Secret =", sanjana_shared)

# STEP 7: Priya Shared Secret
print("\n\nSTEP 7: PRIYA CALCULATES SHARED SECRET")

priya_shared = pow(sanjana_public, priya_private, p)

print("\nPriya Shared Secret =", sanjana_public, "^", priya_private, "mod", p)

print("\nPriya Shared Secret =", priya_shared)

# STEP 8: Verification
print("\n\nSTEP 8: VERIFICATION")

print("\nSanjana Shared Secret =", sanjana_shared)
print("\nPriya Shared Secret =", priya_shared)

if sanjana_shared == priya_shared:
    print("\nSUCCESS!")
    print("Both Sanjana and Priya have the SAME secret key.")
    print("Shared Secret Key =", sanjana_shared)
else:
    print("\nKey Exchange Failed!")

print("\n" + "=" * 60)
print("          PROGRAM COMPLETED")
print("\n" + "=" * 60)

```

OUTPUT:

```

IDLE Shell 3.10.6
File Edit Shell Debug Options Window Help
Python 3.10.6 (tags/v3.10.6:9c7b4bd, Aug 1 2022, 21:53:49) [MSC v.1932 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/ADMIN/Desktop/TYCS_T110_Prac_4_DS/Diffie_Hellman_CL.py =====
DIFFIE-HELLMAN KEY EXCHANGE
=====
STEP 1: PUBLIC PARAMETERS
Enter prime number (p): 23
Enter generator (g): 5

p = 23
g = 5

STEP 2: PRIVATE KEYS
Enter Sanjana's private key: 6
Enter Priya's private key: 15

Sanjana's Private Key = 6
Priya's Private Key = 15

STEP 3: SANJANA GENERATES PUBLIC KEY
Formula:
Sanjana Public Key = g^a mod p
Calculation:
Sanjana Public Key = 5 ^ 6 mod 23
Sanjana Public Key = 8

STEP 4: PRIYA GENERATES PUBLIC KEY
Formula:
Priya Public Key = g^b mod p
Calculation:
Priya Public Key = 5 ^ 15 mod 23
Priya Public Key = 19

STEP 5: PUBLIC KEY EXCHANGE

```

```
STEP 6: SANJANA CALCULATES SHARED SECRET
Sanjana Shared Secret = 19 ^ 6 mod 23
Sanjana Shared Secret = 2

STEP 7: PRIYA CALCULATES SHARED SECRET
Priya Shared Secret = 8 ^ 15 mod 23
Priya Shared Secret = 2

STEP 8: VERIFICATION
Sanjana Shared Secret = 2
Priya Shared Secret = 2

SUCCESS!
Both Sanjana and Priya have the SAME secret key.
Shared Secret Key = 2

=====
PROGRAM COMPLETED
=====
>>>
```



GUI CODE:

```
# =====
# DIFFIE-HELLMAN KEY EXCHANGE - GUI
# =====

import tkinter as tk
from tkinter import messagebox
# FUNCTIONS
def generate_public_keys():
    try:
        p = int(p_entry.get())
        g = int(g_entry.get())

        sanjana_private = int(sanjana_private_entry.get())
        priya_private = int(priya_private_entry.get())

        if p <= 1:
            messagebox.showerror("Error", "p must be greater than 1.")
            return

        if g <= 0:
            messagebox.showerror("Error", "g must be greater than 0.")
            return

        if sanjana_private <= 0 or priya_private <= 0:
            messagebox.showerror(
                "Error",
                "Private keys must be positive numbers."
            )
            return

        # Public keys
        sanjana_public = pow(g, sanjana_private, p)
        priya_public = pow(g, priya_private, p)

        sanjana_public_entry.config(state="normal")
        sanjana_public_entry.delete(0, tk.END)
        sanjana_public_entry.insert(0, str(sanjana_public))
        sanjana_public_entry.config(state="readonly")

        priya_public_entry.config(state="normal")
        priya_public_entry.delete(0, tk.END)
        priya_public_entry.insert(0, str(priya_public))
        priya_public_entry.config(state="readonly")

        # Show calculation
        output_text.delete("1.0", tk.END)

        output_text.insert(
            tk.END,
            "STEP 1: PUBLIC PARAMETERS\n\n"
        )

        output_text.insert(
            tk.END,
            f"p = {p}\n"
            f"g = {g}\n\n"
        )

        output_text.insert(
            tk.END,
            "STEP 2: PRIVATE KEYS\n\n"
        )

        output_text.insert(
            tk.END,
            f"Sanjana's Private Key = {sanjana_private}\n"
            f"Priya's Private Key = {priya_private}\n\n"
        )

        output_text.insert(
            tk.END,
            "STEP 3: SANJANA GENERATES PUBLIC KEY\n\n"
        )

        output_text.insert(
            tk.END,
            "Formula:\n"
            "Sanjana Public Key = g^a mod p\n\n"
        )

        output_text.insert(
```

```

        tk.END,
        f"Calculation:\n"
        f"Sanjana Public Key = {g} ^ a "
        f"({sanjana_private} mod {p})\n\n"
    )

    output_text.insert(
        tk.END,
        f"Sanjana Public Key = {sanjana_public}\n\n\n"
    )

    output_text.insert(
        tk.END,
        "STEP 4: PRIYA GENERATES PUBLIC KEY\n\n"
    )

    output_text.insert(
        tk.END,
        "Formula:\n"
        f"Priya Public Key = g^b mod p\n\n"
    )

    output_text.insert(
        tk.END,
        f"Calculation:\n"
        f"Priya Public Key = {g} ^ a "
        f"({priya_private} mod {p})\n\n"
    )

    output_text.insert(
        tk.END,
        f"Priya Public Key = {priya_public}\n\n\n"
    )

    output_text.insert(
        tk.END,
        "STEP 5: PUBLIC KEY EXCHANGE\n\n"
    )

    output_text.insert(
        tk.END,
        f"Sanjana -> Priya : {sanjana_public}\n"
        f"Priya -> Sanjana : {priya_public}\n\n"
        "Private keys are NOT exchanged.\n"
    )

    messagebox.showinfo(
        "Success",
        "Public Keys Generated Successfully!"
    )

except ValueError:
    messagebox.showerror(
        "Error",
        "Please enter valid numbers."
    )

def generate_shared_key():
    try:
        p = int(p_entry.get())

        sanjana_private = int(
            sanjana_private_entry.get()
        )

        priya_private = int(
            priya_private_entry.get()
        )

        sanjana_public = int(
            sanjana_public_entry.get()
        )

        priya_public = int(
            priya_public_entry.get()
        )

        # Shared keys
        sanjana_shared = pow(
            priya_public,
            sanjana_private,
            p
        )

        priya_shared = pow(
            sanjana_public,
            priya_private,
            p
        )

        # Display shared keys
        sanjana_shared_entry.config(state="normal")
        sanjana_shared_entry.delete(0, tk.END)
        sanjana_shared_entry.insert(
            0,
            str(sanjana_shared)
        )
        sanjana_shared_entry.config(state="readonly")

        priya_shared_entry.config(state="normal")
        priya_shared_entry.delete(0, tk.END)
        priya_shared_entry.insert(
            0,
            str(priya_shared)
        )
        priya_shared_entry.config(state="readonly")

    output_text.insert(
        tk.END,
        "\n\n"
        "STEP 6: SANJANA CALCULATES SHARED SECRET\n\n"
    )

    output_text.insert(
        tk.END,
        "Sanjana Shared Secret = "
        f"({priya_public} ^ a "
        f"({sanjana_private} mod {p})\n\n"
    )

    output_text.insert(

```

```

        tk.END,
        f"Sanjana Shared Secret = "
        f"{sanjana_shared}\n\n"
    )

    output_text.insert(
        tk.END,
        "STEP 7: PRIYA CALCULATES SHARED SECRET\n\n"
    )

    output_text.insert(
        tk.END,
        f"Priya Shared Secret = "
        f"{(sanjana_public ^ p)}\n\n"
        f"{(priya_private) mod (p)}\n\n"
    )

    output_text.insert(
        tk.END,
        f"Priya Shared Secret = "
        f"{priya_shared}\n\n"
    )

    messagebox.showinfo(
        "Success",
        "Shared Secret Keys Generated!"
    )

except ValueError:
    messagebox.showerror(
        "Error",
        "First generate the Public Keys."
    )

def verify_key():
    try:
        sanjana_shared = int(
            sanjana_shared_entry.get()
        )

        priya_shared = int(
            priya_shared_entry.get()
        )

        output_text.insert(
            tk.END,
            "STEP 8: VERIFICATION\n\n"
        )

        output_text.insert(
            tk.END,
            f"Sanjana Shared Secret = "
            f"{sanjana_shared}\n"
        )

        output_text.insert(
            tk.END,
            f"Priya Shared Secret = "
            f"{priya_shared}\n\n"
        )

        if sanjana_shared == priya_shared:

            output_text.insert(
                tk.END,
                "SUCCESS\n"
            )

            output_text.insert(
                tk.END,
                "Both Sanjana and Priya have "
                "the SAME secret key.\n"
            )

            output_text.insert(
                tk.END,
                f"Shared Secret Key = "
                f"{sanjana_shared}\n"
            )

            messagebox.showinfo(
                "Verification Successful",
                f"Both keys are SAME!\n\n"
                f"Shared Secret Key = "
                f"{sanjana_shared}"
            )

        else:

            output_text.insert(
                tk.END,
                "Key Exchange Failed!\n"
            )

            messagebox.showerror(
                "Verification Failed",
                "Shared keys are different."
            )

    except ValueError:
        messagebox.showerror(
            "Error",
            "Please generate the Shared Keys first."
        )

def clear_all():
    for entry in [
        p_entry,
        g_entry,
        sanjana_private_entry,
        priya_private_entry
    ]:
        entry.delete(0, tk.END)

    for entry in [
        sanjana_public_entry,
        priya_public_entry,
        sanjana_shared_entry,
        priya_shared_entry
    ]:

```

```

        entry.config(state="normal")
        entry.delete(0, tk.END)
        entry.config(state="readonly")

    output_text.delete("1.0", tk.END)

# GUI WINDOW
root = tk.Tk()

root.title("Diffie-Hellman Key Exchange")
root.geometry("900x750")
root.resizable(False, False)

root.configure(bg="#EAF2F8")
# TITLE
title = tk.Label(
    root,
    text="DIFFIE-HELLMAN KEY EXCHANGE",
    font=("Arial", 22, "bold"),
    bg="#F618D",
    fg="white",
    pady=15
)

title.pack(fill="x")
# INPUT FRAME
input_frame = tk.Frame(
    root,
    bg="#EAF2F8"
)

input_frame.pack(
    padx=20,
    pady=15
)

# Public Parameters

tk.Label(
    input_frame,
    text="Public Parameters",
    font=("Arial", 14, "bold"),
    bg="#EAF2F8"
).grid(row=0, column=0, columnspan=2, pady=8)

tk.Label(
    input_frame,
    text="Prime Number (p):",
    font=("Arial", 11),
    bg="#EAF2F8"
).grid(row=1, column=0, padx=10, pady=5, sticky="e")

p_entry = tk.Entry(
    input_frame,
    font=("Arial", 11),
    width=20
)

p_entry.grid(row=1, column=1, pady=5)

tk.Label(
    input_frame,
    text="Generator (g):",
    font=("Arial", 11),
    bg="#EAF2F8"
).grid(row=2, column=0, padx=10, pady=5, sticky="e")

g_entry = tk.Entry(
    input_frame,
    font=("Arial", 11),
    width=20
)

g_entry.grid(row=2, column=1, pady=5)

# Private Keys

tk.Label(
    input_frame,
    text="PRIVATE KEYS",
    font=("Arial", 14, "bold"),
    bg="#EAF2F8"
).grid(
    row=3,
    column=0,
    columnspan=2,
    pady=(15, 8)
)

tk.Label(
    input_frame,
    text="Sanjana Private Key:",
    font=("Arial", 11),
    bg="#EAF2F8"
).grid(row=4, column=0, padx=10, pady=5, sticky="e")

sanjana_private_entry = tk.Entry(
    input_frame,
    font=("Arial", 11),
    width=20
)

sanjana_private_entry.grid(row=4, column=1, pady=5)

tk.Label(
    input_frame,
    text="Priya Private Key:",
    font=("Arial", 11),
    bg="#EAF2F8"
).grid(row=5, column=0, padx=10, pady=5, sticky="e")

priya_private_entry = tk.Entry(
    input_frame,
    font=("Arial", 11),
    width=20
)

```

```

priya_private_entry.grid(row=5, column=1, pady=5)
# PUBLIC KEY FRAME
key_frame = tk.Frame(
    root,
    bg="#EAF2F8"
)

key_frame.pack(pady=10)

tk.Label(
    key_frame,
    text="Sanjana Public Key:",
    font=("Arial", 11, "bold"),
    bg="#EAF2F8"
).grid(row=0, column=0, padx=10)

sanjana_public_entry = tk.Entry(
    key_frame,
    font=("Arial", 11),
    width=18,
    state="readonly"
)

sanjana_public_entry.grid(row=0, column=1, padx=10)

tk.Label(
    key_frame,
    text="Priya Public Key:",
    font=("Arial", 11, "bold"),
    bg="#EAF2F8"
).grid(row=0, column=2, padx=10)

priya_public_entry = tk.Entry(
    key_frame,
    font=("Arial", 11),
    width=18,
    state="readonly"
)

priya_public_entry.grid(row=0, column=3, padx=10)
# SHARED KEY FRAME
shared_frame = tk.Frame(
    root,
    bg="#EAF2F8"
)

shared_frame.pack(pady=10)

tk.Label(
    shared_frame,
    text="Sanjana Shared Key:",
    font=("Arial", 11, "bold"),
    bg="#EAF2F8"
).grid(row=0, column=0, padx=10)

sanjana_shared_entry = tk.Entry(
    shared_frame,
    font=("Arial", 11),
    width=18,
    state="readonly"
)

sanjana_shared_entry.grid(row=0, column=1, padx=10)

tk.Label(
    shared_frame,
    text="Priya Shared Key:",
    font=("Arial", 11, "bold"),
    bg="#EAF2F8"
).grid(row=0, column=2, padx=10)

priya_shared_entry = tk.Entry(
    shared_frame,
    font=("Arial", 11),
    width=18,
    state="readonly"
)

priya_shared_entry.grid(row=0, column=3, padx=10)
# BUTTONS
button_frame = tk.Frame(
    root,
    bg="#EAF2F8"
)

button_frame.pack(pady=10)

tk.Button(
    button_frame,
    text="Generate Public Keys",
    font=("Arial", 11, "bold"),
    width=20,
    command=generate_public_keys
).grid(row=0, column=0, padx=5)

tk.Button(
    button_frame,
    text="Generate Shared Key",
    font=("Arial", 11, "bold"),
    width=20,
    command=generate_shared_key
).grid(row=0, column=1, padx=5)

tk.Button(
    button_frame,
    text="Verify Key",
    font=("Arial", 11, "bold"),
    width=15,
    command=verify_key
).grid(row=0, column=2, padx=5)

tk.Button(
    button_frame,
    text="Clear",

```

```

font=("Arial", 11, "bold"),
width=12,
command=clear_all
).grid(row=0, column=3, padx=5)

# OUTPUT AREA
tk.Label(
    root,
    text="STEP-BY-STEP OUTPUT",
    font=("Arial", 14, "bold"),
    bg="#EAF2F8"
).pack(pady=(5, 5))

output_text = tk.Text(
    root,
    height=13,
    width=105,
    font=("Consolas", 10),
    wrap="word"
)

output_text.pack(
    padx=20,
    pady=5
)

# START GUI
root.mainloop()

```

OUTPUT:

DIFFIE-HELLMAN KEY EXCHANGE

PUBLIC PARAMETERS

Prime Number (p):

Generator (g):

PRIVATE KEYS

Sanjana Private Key:

Priya Private Key:

Sanjana Public Key:

Priya Public Key:

Sanjana Shared Key:

Priya Shared Key:

Generate Public Keys

Generate Shared Key

Clear

Exit Full Screen

STEP 1: PUBLIC PARAMETERS

p = 23
g = 5

STEP 2: PRIVATE KEYS

Sanjana's Private Key = 6
Priya's Private Key = 15

STEP 3: SANJANA GENERATES PUBLIC KEY

Formula:
Sanjana Public Key = $g^a \text{ mod } p$

i

Verification Successful

Both keys are SAME!
Shared Secret Key = 2

OK